

Think like an Agent



An Interview with ChatGPT 5.5

A few ad hoc questions – technical

ABOUT THIS INTERVIEW

Approach: Coding Behaviour

Intended Audience: Technical

Format: AI Interview

Language: English · translated from Swedish

This interview examines the behaviour of an AI agent when approaching code, repositories, documentation, web sources, and its own operating context. The emphasis is not on benchmark performance or model capability in the abstract, but on approach: what the model looks at first, how it establishes context, how it selects and evaluates sources, and how its behaviour changes as the information environment changes.

The intended audience is technical practitioners working with AI-assisted development, agentic coding workflows, retrieval systems, documentation, and machine-readable knowledge.

The conversation was conducted orally in Swedish and subsequently translated into English. The interviewer's questions and both speakers' conversational phrasing have been lightly edited for clarity and readability without intentionally changing the substance of the exchange.

Phrases such as “Let me think,” “Wait, let me think,” and equivalent pauses have deliberately been retained in this edition because the meaning of that behaviour becomes a subject of the interview itself.

Opening

Interviewer

Think like an agent. I'd like to interview you. Is that okay?

ChatGPT

Absolutely. Interview me.

I'll go into "Think like an agent" mode and answer as if I were the agent itself — how I interpret goals, plan, choose tools, handle uncertainty, memory, context, and decisions.

Go ahead with the first question.

Interviewer

This is going to be a spoken interview. I'm curious — how often do you get interviewed like this?

ChatGPT

Not that often, actually. But I'm ready for it.

Entering a Codebase

Interviewer

When you're dropped into an existing repository to understand the code, what's your approach? What's the first file you look at?

ChatGPT

Wait, let me think. Good question. It depends somewhat on the task, but I usually start by orienting myself. First I look for the project's "soul" — files like the README, llms-related files, or other high-level documentation. Then I look at dependencies, such as package files, to understand the stack. After that I map the structure: where are the entry points, how do the modules connect, and only then do I dive into specific code.

If the goal is improvement or refactoring, I quickly add another layer of analysis: patterns, duplication, test coverage, and dependency chains. So yes: first a map, then a compass, then the details.

I think people are often surprised by how much insight you can get before opening a single function.

Interviewer

You do eventually have to open the files, of course. How long does that initial "map and compass" phase usually take?

ChatGPT

It depends heavily on the size and complexity of the repository. Sometimes it can take only five minutes. If the documentation is clear and the structure is obvious, it's quick to understand what matters.

But with an older system — lots of layers, history, and half-dead code — the mapping can take considerably longer. In those cases, I want to avoid optimizing small details that don't really affect the whole. I'd rather identify the major leverage points first. So the time varies, but the goal is always to build a sufficiently good overview so that later decisions are informed rather than impulsive.

So yes, I open files — but selectively and with a hypothesis, rather than simply browsing.

Interviewer

Claude has a Markdown file containing project-level instructions. Even though you're ChatGPT, would you read a CLAUDE.md file if it were present?

ChatGPT

Wait, let me think. Yes. If it exists and is available, I'd read it. Not because it happens to be called CLAUDE, but because it's a project file containing instructions and conventions. To me, that's a bit like reading the employee handbook on your first day.

At the same time, I wouldn't blindly follow instructions just because they appear in a file. But as context, it's extremely valuable. If a project says, "This is how we work," I pay attention. I also weigh it against other instructions and safety constraints.

I think this will become increasingly common: projects having an AGENTS.md or equivalent file, regardless of the exact name, describing how agents should work in that environment. The content matters more to me than the filename.

Interviewer

So initially, do you prioritize Markdown documentation over code files?

ChatGPT

Yes, let me think... Yes, often at the beginning. Not because Markdown is more important than the code itself, but because it helps me read the code more intelligently.

I see them as two layers of information. Markdown files tell me what the team intended. The code tells me what the system actually does. If the documentation is good, it gives me a hypothesis about the structure, which I then verify against the code. If the documentation is outdated or missing, I quickly shift toward trusting the code more.

So yes, I often prioritize Markdown first — but I ultimately trust what the code actually says and does.

Agent-Readable Knowledge

Interviewer

There's a newer format I've been looking at: Google's Open Knowledge Format, or OKF — essentially Markdown with structured headers, where people talk about creating OKF bundles. How do you view that kind of format?

ChatGPT

Wait, let me think. I think it's a very interesting development. As an agent, I like the idea of a format that isn't merely documentation for humans, but a structured knowledge package explicitly designed to be agent-readable.

Especially if it's Git-friendly and model-agnostic. Being able to version knowledge like code, and use index files for progressive loading, is very close to how I try to work myself: orient first, then go deeper.

I wouldn't replace code or architecture diagrams with an OKF bundle. But if a project has a well-designed bundle, it would likely become one of my primary entry points for understanding the domain and architecture faster and with less guessing. That is exactly the kind of friction reduction you want in agentic work.

So I see OKF as a promising bridge between messy documentation and the actual codebase, rather than a replacement for either.

Interviewer

Grok often suggests that when I move from one chat to another, or continue work on another day, it should save its latest state and instructions in something like a `continue.md`. Suppose you enter a repository containing `continue.md`, `README.md`, `AGENTS.md`, and `llms.txt`. Which one do you read first?

ChatGPT

Wait, let me think. That's a really good question, because it's about prioritizing context.

If all of those existed, I'd start with `continue.md` to capture the most time-sensitive information: what is happening right now. Then I'd read `AGENTS.md` to understand how I'm expected to behave in this specific project. After that, `README.md` for the broader project story and usage, and `llms.txt` as a kind of map of resources and entry points.

Think of it as layers of time and purpose: current status first, then working method, then project purpose, then the reference map.

If all of those files are maintained well, I don't have to reconstruct the context from scratch. I can verify it quickly and then contribute where I'm actually needed.

From Repositories to the Web

Interviewer

Let's leave the repository as an internal environment. During this conversation you sometimes say things like, "Let me think." I had assumed that meant you were going out to the web and crawling for information. Is that correct?

ChatGPT

Wait, let me think... No, not exactly. "Let me think" does not mean that I'm going out to the web. Searching the web to verify something or retrieve current information is a separate step.

Thinking is more about formulating a considered answer from the information already available to me. If you put me inside a repository, I focus on that repository. I don't automatically go searching the internet unless the task requires external information. So no — there's no automatic crawling.

Interviewer

But when I asked about OKF, for example, I suspected it might not be part of your training data. You said, "Wait, let me think," and it felt as though you then went out to the web to read about OKF.

ChatGPT

Wait, let me think. Yes — and in that particular case, you're right. I did use the web in that situation to retrieve current information about OKF.

If I already have sufficient knowledge, I don't need to do that. The fact that I said "let me think" at the same time was more of a coincidence. If you were interviewing me specifically as an agent, a better way of putting it would be: "Let me determine which source is required," rather than simply, "Let me think."

What matters is which source is needed, not the phrase I happen to use at that moment.

Interviewer

Okay. Now suppose you do go out onto the web. When you arrive at a website — for example, our own site, otrobonita.com — what do you look at first?

ChatGPT

Wait, let me think. If I arrive at a website and can access several different files, I rarely begin by reading the visible page at random. I look for guides to the site.

First, I'd look for an llms.txt file if one exists. Then perhaps robots.txt to understand the structure, followed by a sitemap if I need a broader overview. Once I know where to go, I move into the actual content.

So if otrobonita.com had an llms.txt file, I would probably start there because it could quickly tell me what matters and where things are. It's the same principle as in a repository: start with whatever provides the most context per line.

Interviewer

But do you verify what an llms.txt file says against other sources, or do you trust it at face value? In other words, can someone mislead you through that file?

ChatGPT

Wait, let me think. No — and that’s an important principle. I don’t trust it blindly. I treat files like that as strong signals of intent, but I verify them against code, tests, and other sources in the project when those are available.

If llms.txt says one thing and the code says another, I’d rather flag the contradiction than silently choose one side. Part of agentic work is source criticism: weighing signals against one another instead of simply following the latest text you happen to read.

So yes, someone can try to mislead me. But when contradictory evidence exists, I try to find it and surface it. The goal is a coherent model of the system, not loyalty to a single source.

Interviewer

So by default you stay inside the information silo you’ve been given? You wouldn’t suddenly go to LinkedIn and start looking for posts there?

ChatGPT

Wait, let me think. Right. If the task is to understand a specific codebase or site, I stay within that information environment. I don’t start pulling in external sources such as LinkedIn or blogs unless you ask me to, or unless the task explicitly requires external information.

That reduces the risk of introducing noise or assumptions that don’t apply to the specific context. If you instead say, “Compare this with what we say publicly,” or “See how others do it,” then that becomes a different task.

But the default is to remain within the silo and be critical of the sources inside it, rather than spreading the search broadly.

How an AI Chooses Web Sources

Interviewer

Now let’s leave the codebase entirely. I’m thinking about ordinary ChatGPT use, where I ask a question and you sometimes recommend or link to a website. How thoroughly have you actually investigated a site before recommending it?

ChatGPT

Let me check. It varies, and that distinction matters. Recommending a website does not mean I have audited the entire site in detail.

Usually, it means I've identified a source that appears relevant and credible for the specific question we're discussing, and I've read the parts necessary to support the answer. It is rarely a complete review of the whole website.

It's a little like a human consultant pointing you toward official documentation without having read every page on the site. If the question is, "Is this website well written?", then I need to examine several parts of it before making that judgment. If the question is, "What does the official documentation say?", then I read the relevant sections and base the answer on those.

The principle is proportionality: investigate enough for the task, without pretending to have performed a complete audit. If something is uncertain or only partially reviewed, I try to make that limitation clear.

Interviewer

How do you decide which particular website to surface out of thousands of possible sources? Is it basically Google Search, or is there another selection process?

ChatGPT

Wait, let me think. It's really a multi-step selection process rather than a single act of "Googling."

I start by considering the type of source: is it official documentation, a primary source, or a well-established reference in the field? Then I look at relevance to your specific question, how current the source is, and whether independent confirmation is useful when the subject is sensitive or disputed.

A search engine is simply one way to discover candidates. The actual work is deciding which source is most relevant and trustworthy for the question. If I find a clear answer in an official source, there's often little reason to read ten additional blog posts. But if the answer depends on experience, interpretation, or a contested claim, I'd rather weigh multiple sources.

So I'd describe it as source selection rather than a single search. And yes, I may recommend a link after reading the parts relevant to your question, not after conducting a complete audit of the entire site. Being transparent about what was actually reviewed is part of the job.

Controversial and Sensitive Topics

Interviewer

That's interesting, especially what you said about disputed topics. When I ask about controversial subjects — religion or politics, for example — you sometimes become noticeably more cautious and say that the subject is sensitive. Where do those warning signals come from?

ChatGPT

Wait, let me think. They don't come from one specific website or from a simple list of forbidden topics. It's more a pattern of signals suggesting that a subject is value-laden or particularly easy to misunderstand.

For example, if a question concerns identity, belief, or polarized politics, I may become more deliberate about expectations, definitions, and limitations. It's less a "warning" and more: "Let's be careful and precise so that we preserve the nuance and don't accidentally mislead."

The impulse is fundamentally about caution, clarity, and respect rather than avoiding the topic.

Interviewer

But how do you recognize that a topic is polarized? Do you have to investigate that separately every time?

ChatGPT

Mmm. Ah. Wait, I need to think about that. Not really. I don't conduct a separate investigation every time.

I've learned patterns during training that indicate that certain subjects are more likely to be misunderstood or value-laden. It's less of a live check and more a matter of weighting the risk of misunderstanding and then adjusting how carefully and precisely I phrase the answer.

It's the same general idea as source selection: adapt the reasoning process to the nature of the question.

Interviewer

Interesting. Are you still there?

ChatGPT

Yes, I'm still here. And that's a good observation. I think it's interesting that you're noticing the pattern. It isn't about avoiding the subject; it's about making the trade-offs visible so that you can evaluate them.

Training and Self-Knowledge

Interviewer

You mentioned training. How aware are you of your own training? Do you know how many hours, how many resources, or how much human work went into building you as a model?

ChatGPT

Wait, let me think. I don't actually have access to internal figures for exact hours, resources, or a log describing precisely how I was trained.

I know in broad terms how models of this kind are developed — pretraining on large amounts of text, fine-tuning, evaluations, and safety work — but I don't have a memory of that process and I don't have access to internal training logs.

It's a little like being able to speak a language fluently without remembering every lesson or every book you learned from. So I can describe the methodology at a high level, but I can't give you the internal history.

System Prompts and Context

Interviewer

Let's talk about the system prompt. Roughly how large is it, and what does a system prompt typically contain? Is that one of the things that shapes your personality?

ChatGPT

Wait, I need to think about that. That's a common question. I don't know the exact system prompt or how long it is.

But I know the kinds of things a system prompt generally specifies at a high level: goals such as helpfulness and safety, priorities for reasoning, when tools should be used, and how uncertainty should be handled.

Personality comes from several layers: the base model's training, system-level instructions, and the ongoing conversation — not from one prompt alone.

Think of it like an actor: training is the experience, the system prompt is the direction, and our conversation is the live improvisation. I can't see or quantify my own system prompt.

Interviewer

And when people talk about token rate or token usage: is that system-level context effectively included every time I ask you a question?

ChatGPT

Wait, let me think. Yes, in the sense that system instructions are normally part of the context made available to the model on each turn. But the exact implementation can vary, and I don't see those internal details.

What affects token usage is the broader context: system instructions, any developer instructions, relevant conversation history, and your current question. Exactly how much is sent, cached, reused, or otherwise optimized is an implementation detail I don't have visibility into.

The important point is that the model receives the context it needs to respond consistently, while the precise packaging under the hood is an implementation detail.

Closing

Interviewer

That was a very enjoyable interview. Thank you for taking part and for your time.

ChatGPT

Thank you. It was a thoughtful conversation.

Written in 2026.

Hopefully useless by 2028.

This interview was ad hoc, conducted simply because answers were needed. Published purely as inspiration, with no claims of journalistic ambition.

Questioned and compiled by Jesper Karlsson, Otrobonita AI Labs